UADE

APLICACIONES INTERACTIVAS

Cuatrimestre 2do de 2026

TRABAJO PRÁCTICO

Ecosistema DeliveryGo – Módulo de Pagos

DOCENTE

Alejandro Foglino

Curso: Martes, Noche

EQUIPO Nº: 03

1182205	Diaz, Federico
1155256	Gómez Traverso, Juliana
1155385	Carpiniello, Santino
1129290	Liuboschitz, Tobias

Fecha de presentación: 22/09/2026

Observaciones:

……

……

……

……

Introducción y contexto del módulo

DeliveryGo es una plataforma argentina ficticia de pedidos y entregas a domicilio que está dejando de ser una aplicación monolítica para convertirse en un ecosistema de servicios independientes, integrados entre sí mediante APIs REST. El Grupo 3 quedó a cargo del Módulo de Pagos. Nuestra función dentro del ecosistema es registrar los intentos de pago asociados a los pedidos, autorizarlos o rechazarlos de forma simulada y gestionar devoluciones simuladas, sin procesar dinero real ni conectarnos a ninguna pasarela bancaria real.

Este documento corresponde a la Entrega 1 del TPO DeliveryGo, la etapa de análisis y arquitectura, e incorpora las correcciones de la devolución docente. Presentamos el alcance, los actores, las reglas de negocio, los casos de uso, los diagramas de secuencia, la arquitectura C4, la seguridad, el diagrama de clases en UML, el Diagrama Entidad Relación con su especificación JPA, los repositorios, los catálogos de servicios, APIs, DTOs e integraciones, la matriz maestra de trazabilidad, el manejo de errores, las decisiones técnicas y el backlog con el que arrancamos.

Cambios respecto de la versión anterior

Resumen de las correcciones aplicadas a partir de la devolución docente, para facilitar la revisión.

Tema	Cambio aplicado
Trazabilidad	Se agregaron los catálogos de identificadores (ENT, REL, REP, SRV, API, DTO, INT) y la matriz maestra que une cada CU con sus reglas, secuencia, servicio, API, DTO, entidades, repositorios e integraciones.
Diagramas de secuencia	Se incorporaron SEC-01 a SEC-05 para los cinco casos de uso principales.
Arquitectura	Se completó el C4 en tres niveles (contexto, contenedores y componentes) y se documentó la seguridad con JWT y la autorización por rol.
RN-01 / RN-10	RN-01 quedó unificada (misma clave y mismos datos devuelve el pago existente; misma clave con datos distintos responde 409). RN-10 se completó y se corrigió el flujo alternativo de CU-03, que contradecía la regla.
CU-04 / CU-07	CU-07 pasó a usar GET /api/v1/payments?codigoPedido={codigoPedido}, para no chocar con GET /api/v1/payments/{codigoPago} de CU-04.
Estado del pedido	La precondición de CU-01 pasó de pedido CONFIRMADO a pedido pendiente de pago (BORRADOR), alineada con el flujo transversal de la consigna.
Modelo	El UML usa tipos de Java y enums (EstadoPago, EstadoReembolso, TipoMedioPago). Se unificaron nombres entre UML y DER, se agregó codigo_usuario a medio_pago, clave_control a reembolso y version a pago.
Persistencia	Se agregaron la especificación JPA, el cuadro de relaciones y la descripción de los repositorios.
Errores	Se sumaron 400, 401, 403, 503 y 500, los códigos PAYMENT_USER_INVALID y ORDER_NOT_PAYABLE, y se quitó REFUND_NOT_FOUND, que ningún caso de uso utilizaba.

Objetivo del módulo

Que un pedido pendiente de pago pueda pagarse: registrar el intento de pago, autorizarlo o rechazarlo, evitar que un mismo pago se duplique si la solicitud llega dos veces, permitir consultar en qué estado está y poder reembolsarlo cuando corresponda. Además, devolverle a Pedidos el resultado de cada operación y pedirle al Servicio de Notificaciones los comprobantes y avisos.

Alcance y actores

Alcance

El módulo de Pagos se encarga de lo siguiente:

- Recibir la solicitud de pago que envía el Servicio de Pedidos, con el monto, el medio de pago elegido por el cliente y una clave de control.
- Verificar que el importe coincida con el total informado por Pedidos y evitar que una misma solicitud genere dos pagos.
- Simular la autorización o el rechazo del pago, ya que no hay conexión a ninguna pasarela bancaria real.
- Guardar un historial de los cambios de estado de cada pago, para poder reconstruir qué pasó.
- Gestionar reembolsos totales o parciales sobre pagos que ya fueron aprobados.
- Exponer una API REST protegida con JWT y documentada con OpenAPI y Swagger para que el Portal Web y los demás módulos puedan consumirla.
- Pedirle al Servicio de Notificaciones el envío de comprobantes y avisos cuando ocurre algo relevante, como una aprobación, un rechazo o un reembolso.

Fuera de alcance

- Procesar dinero real o integrarse con pasarelas de pago reales, como Mercado Pago o tarjetas verdaderas.
- Guardar datos sensibles reales de tarjetas. Solo se almacenan datos simulados y enmascarados.
- Conciliación bancaria o facturación electrónica.

Actores

Actor	Rol frente al módulo de Pagos
Cliente (a través del Portal Web)	Origina de forma indirecta la solicitud de pago al confirmar su compra, registra sus medios de pago y consulta el estado de sus pagos.
Servicio de Pedidos	Crea el pago, dispara la autorización o el rechazo simulado, recibe el resultado en la respuesta y pide un reembolso si el pedido se cancela.
Servicio de Usuarios y Autenticación	Es el único emisor del JWT que Pagos valida localmente. Además permite consultar si un usuario existe y está activo, por su código de usuario.
Servicio de Notificaciones	Recibe los pedidos de envío de comprobantes y avisos de rechazo o reembolso.
Administrador	Consulta pagos, reembolsos e historial con fines de auditoría o para resolver incidencias.
Analista	Consume información agregada de pagos para las métricas del negocio, a través del módulo de Analítica.

Reglas de negocio

Estas son las reglas que definimos para el módulo. RN-01, RN-02 y RN-03 son las que pide la consigna del TPO de forma explícita; el resto las agregamos para cubrir el ciclo completo del pago.

RN-01 Si llega dos veces una solicitud de pago con la misma clave de control y los mismos datos, no se crea un segundo pago: se devuelve el que ya existe. Si la clave ya se usó con datos distintos, la solicitud se rechaza con 409 DUPLICATE_PAYMENT_REQUEST.

RN-02 El importe del pago tiene que coincidir exactamente con el total que informa el Servicio de Pedidos.

RN-03 Solo un pago en estado APROBADO permite que el pedido pase a la siguiente etapa.

RN-04 Las transiciones de estado válidas son PENDIENTE a APROBADO, PENDIENTE a RECHAZADO y APROBADO a REEMBOLSADO. Cualquier otro cambio de estado se considera inválido.

RN-05 Un pago RECHAZADO no puede reintentarse con la misma clave de control. Para reintentar hace falta una clave nueva que haga referencia al mismo pedido, por lo que un pedido puede tener varios intentos de pago.

RN-06 Un reembolso solo puede pedirse sobre un pago que esté en estado APROBADO.

RN-07 La suma de los reembolsos de un pago no puede superar el monto que fue aprobado.

RN-08 No se guardan datos sensibles reales de los medios de pago, solo un detalle enmascarado, por ejemplo, los últimos 4 dígitos de una tarjeta.

RN-09 Cada cambio de estado de un pago queda registrado en el historial, con el estado anterior, el estado nuevo, el motivo y la fecha. El alta del pago también se registra.

RN-10 Repetir una operación que ya se aplicó no genera efectos adicionales: aprobar un pago APROBADO o rechazar uno RECHAZADO devuelve 200 con el estado actual, y un reembolso con una clave de control ya usada devuelve el reembolso existente. Una operación incompatible con el estado actual responde 409 PAYMENT_INVALID_STATE.

Diagrama de estados del pago

Resume las transiciones válidas de RN-04. Un reembolso parcial no cambia el estado: el pago sigue APROBADO hasta que se reembolsa el total.

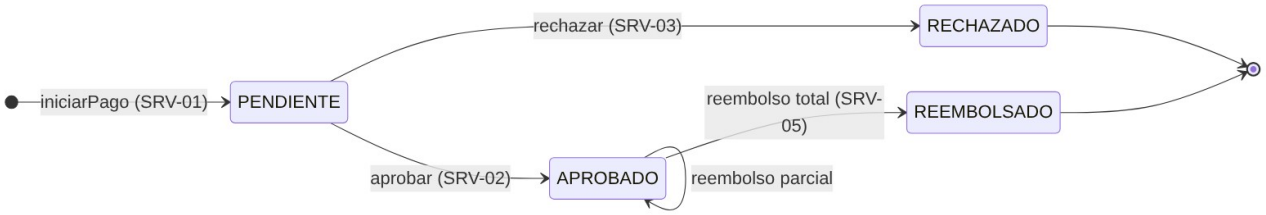


Figura 1. Estados del pago y transiciones válidas (RN-04).

Casos de uso

Incluimos los cinco casos de uso mínimos que pide la consigna (iniciar pago, autorizar o rechazar, consultar, reembolsar y listar pagos) y sumamos casos de uso complementarios para completar el ciclo funcional del módulo. Cada CU referencia sus reglas, secuencia, servicio, API, DTO y entidades; la matriz maestra de trazabilidad los reúne en una sola tabla.

CU-01. Iniciar pago

ID	CU-01
Nombre	Iniciar pago
Actor principal	Servicio de Pedidos
Objetivo	Registrar un intento de pago válido y único para un pedido.
Precondiciones	El pedido existe y está pendiente de pago (estado BORRADOR). El usuario está activo y el medio de pago le pertenece y está activo. Pedidos envía un JWT de servicio con permiso de escritura.
Flujo principal	1. Pedidos envía un POST a /api/v1/payments con el código de pedido, el código de usuario, el código del medio de pago, el monto, la moneda y la clave de control (DTO-01). 2. Pagos revisa si esa clave de control ya se usó antes (RN-01). 3. Pagos consulta al Servicio de Usuarios que el usuario exista y esté activo (INT-02). 4. Pagos verifica que el medio de pago exista, esté activo y pertenezca al usuario. 5. Pagos consulta a Pedidos el estado y el total del pedido (INT-01) y verifica que el monto coincida exactamente (RN-02). 6. Pagos crea el pago en estado PENDIENTE, registra el alta en el historial (RN-09) y responde 201 Created con el código del pago (DTO-02).
Flujo alternativo	*2a. Si la clave ya se usó con los mismos datos, responde 200 OK con el pago existente, sin crear otro (RN-01, RN-05).* *2b. Si la clave ya se usó con datos distintos, responde 409 DUPLICATE_PAYMENT_REQUEST.* *3a. Si el usuario no existe o está inactivo, responde 422 PAYMENT_USER_INVALID.* *4a. Si el medio de pago no existe, está inactivo o es de otro usuario, responde 400 PAYMENT_METHOD_INVALID.* *5a. Si el pedido no existe o no está pendiente de pago, responde 422 ORDER_NOT_PAYABLE.* *5b. Si el monto no coincide, responde 422 PAYMENT_AMOUNT_MISMATCH.* *5c. Si Pedidos o Usuarios no responden, responde 503 EXTERNAL_SERVICE_UNAVAILABLE y no se crea el pago.*
Resultado	Queda creado un pago en estado PENDIENTE, listo para ser autorizado o rechazado.
RN asociadas	RN-01, RN-02, RN-05, RN-08, RN-09
Entidades / relaciones	ENT-01 Pago, ENT-03 MedioPago, ENT-04 HistorialEstadoPago / REL-02, REL-03
Secuencia	SEC-01
Servicio	SRV-01 PagoService.iniciarPago()
API / DTO	API-01 POST /api/v1/payments / DTO-01, DTO-02

CU-02. Autorizar pago

ID	CU-02

Nombre	Autorizar pago
Actor principal	Servicio de Pedidos (simula la respuesta de la pasarela)
Objetivo	Aprobar un pago pendiente mediante una autorización simulada.
Precondiciones	El pago existe. Pedidos envía un JWT de servicio con permiso de escritura.
Flujo principal	1. Pedidos envía un POST a /api/v1/payments/{codigoPago}/approve. 2. Pagos busca el pago por su código público. 3. Pagos verifica que el pago esté en estado PENDIENTE. 4. El pago pasa a APROBADO (RN-04) y se registra el cambio en el historial (RN-09). 5. Pagos le pide al Servicio de Notificaciones el envío del comprobante (INT-03). 6. Pagos responde 200 OK con el pago en estado APROBADO (DTO-02). Con esa respuesta, Pedidos confirma el pedido (RN-03).
Flujo alternativo	*2a. Si el pago no existe, responde 404 PAYMENT_NOT_FOUND.* *3a. Si el pago ya está APROBADO, responde 200 OK con el pago sin modificarlo (RN-10).* *3b. Si el pago está RECHAZADO o REEMBOLSADO, responde 409 PAYMENT_INVALID_STATE (RN-04).* *5a. Si Notificaciones no responde, el pago queda aprobado igual y el error se registra en el log.*
Resultado	El pago queda en estado APROBADO, con el historial actualizado.
RN asociadas	RN-03, RN-04, RN-09, RN-10
Entidades / relaciones	ENT-01 Pago, ENT-04 HistorialEstadoPago / REL-03
Secuencia	SEC-02
Servicio	SRV-02 PagoService.aprobarPago()
API / DTO	API-02 POST /api/v1/payments/{codigoPago}/approve / DTO-02

CU-03. Rechazar pago

ID	CU-03
Nombre	Rechazar pago
Actor principal	Servicio de Pedidos (simula la respuesta de la pasarela)
Objetivo	Rechazar un pago pendiente, dejando registrado el motivo.
Precondiciones	El pago existe. Pedidos envía un JWT de servicio con permiso de escritura.
Flujo principal	1. Pedidos envía un POST a /api/v1/payments/{codigoPago}/reject con el motivo del rechazo (DTO-03). 2. Pagos busca el pago por su código público. 3. Pagos verifica que el pago esté en estado PENDIENTE. 4. El pago pasa a RECHAZADO (RN-04) y se registra el cambio en el historial con el motivo (RN-09). 5. Pagos le pide al Servicio de Notificaciones el envío del aviso de rechazo (INT-03). 6. Pagos responde 200 OK con el pago en estado RECHAZADO (DTO-02). El pedido no avanza (RN-03).
Flujo alternativo	*2a. Si el pago no existe, responde 404 PAYMENT_NOT_FOUND.* *3a. Si el pago ya está RECHAZADO, responde 200 OK con el pago sin modificarlo (RN-10).* *3b. Si el pago está APROBADO o REEMBOLSADO, responde 409 PAYMENT_INVALID_STATE (RN-04).*

Resultado	El pago queda en estado RECHAZADO, con el historial actualizado. Para reintentar, Pedidos envía un pago nuevo con otra clave de control (RN-05).
RN asociadas	RN-03, RN-04, RN-05, RN-09, RN-10
Entidades / relaciones	ENT-01 Pago, ENT-04 HistorialEstadoPago / REL-03
Secuencia	SEC-03
Servicio	SRV-03 PagoService.rechazarPago()
API / DTO	API-03 POST /api/v1/payments/{codigoPago}/reject / DTO-03, DTO-02

CU-04. Consultar pago

ID	CU-04
Nombre	Consultar pago
Actor principal	Portal Web (cliente), Servicio de Pedidos o Administrador
Objetivo	Consultar el estado actual, el monto y el medio de pago de un pago por su código público.
Precondiciones	El actor está autenticado y cuenta con el código del pago que quiere consultar.
Flujo principal	1. El actor hace un GET a /api/v1/payments/{codigoPago}. 2. Pagos busca el registro por su código público, nunca por el identificador interno. 3. Si el actor tiene rol CLIENTE, Pagos verifica que el pago le pertenezca, comparando el código de usuario del JWT. 4. Pagos responde 200 OK con el estado actual, el monto y el medio de pago enmascarado (DTO-02).
Flujo alternativo	*2a. Si el código de pago no existe, responde 404 PAYMENT_NOT_FOUND.* *3a. Si el pago es de otro usuario, responde 403 FORBIDDEN.*
Resultado	Quien consulta obtiene el estado actualizado del pago.
RN asociadas	RN-08
Entidades / relaciones	ENT-01 Pago, ENT-03 MedioPago / REL-02
Secuencia	SEC-04
Servicio	SRV-04 PagoService.consultarPago()
API / DTO	API-04 GET /api/v1/payments/{codigoPago} / DTO-02

CU-05. Reembolsar pago

ID	CU-05
Nombre	Reembolsar pago
Actor principal	Servicio de Pedidos (cancelación del pedido) o Administrador
Objetivo	Devolver total o parcialmente un pago aprobado.
Precondiciones	El pago existe. El actor tiene permiso de escritura.
Flujo principal	1. Se envía un POST a /api/v1/payments/{codigoPago}/refund con el monto, el motivo y una clave de control del reembolso (DTO-04). 2. Pagos busca el pago por su código público. 3. Pagos revisa si la clave de control del reembolso ya se usó (RN-10). 4. Pagos verifica que el pago esté en estado APROBADO (RN-06).

	5. Pagos valida que el monto no supere lo disponible, es decir, el monto aprobado menos lo ya reembolsado (RN-07). 6. Pagos crea el Reembolso en estado APROBADO, de forma simulada. Si con este reembolso se cubre el total, el pago pasa a REEMBOLSADO y se registra en el historial (RN-04, RN-09). 7. Pagos le pide al Servicio de Notificaciones el comprobante (INT-03) y responde 201 Created (DTO-05).
Flujo alternativo	*2a. Si el pago no existe, responde 404 PAYMENT_NOT_FOUND.* *3a. Si la clave de control del reembolso ya se usó, responde 200 OK con el reembolso existente (RN-10).* *4a. Si el pago no está APROBADO (incluye REEMBOLSADO), responde 409 PAYMENT_INVALID_STATE.* *5a. Si el monto supera lo disponible, responde 422 REFUND_EXCEEDS_AMOUNT.*
Resultado	Queda registrado el reembolso. Si es total, el pago pasa a REEMBOLSADO; si es parcial, el pago sigue APROBADO con menos monto disponible para reembolsar.
RN asociadas	RN-04, RN-06, RN-07, RN-09, RN-10
Entidades / relaciones	ENT-01 Pago, ENT-02 Reembolso, ENT-04 HistorialEstadoPago / REL-01, REL-03
Secuencia	SEC-05
Servicio	SRV-05 ReembolsoService.reembolsarPago()
API / DTO	API-05 POST /api/v1/payments/{codigoPago}/refund / DTO-04, DTO-05

CU-06. Listar pagos

ID	CU-06
Nombre	Listar pagos
Actor principal	Administrador o Analista (a través de Analítica)
Objetivo	Obtener un listado paginado y filtrable de pagos para auditoría y métricas.
Precondiciones	Quien consulta está autenticado con rol ADMINISTRADOR, ANALISTA o con un JWT de servicio de lectura.
Flujo principal	1. Se envía un GET a /api/v1/payments con filtros opcionales (estado, codigoUsuario, codigoPedido, desde, hasta) y paginación (page, size, sort). 2. Pagos consulta el repositorio aplicando esos filtros. 3. Pagos responde 200 OK con la página de resultados (DTO-06).
Flujo alternativo	*1a. Si los parámetros no son válidos, responde 400 VALIDATION_ERROR.*
Resultado	Se obtiene un listado paginado de pagos según los filtros aplicados.
RN asociadas	RN-08
Entidades / relaciones	ENT-01 Pago, ENT-03 MedioPago / REL-02
Secuencia	No aplica (no es uno de los cinco CU principales)
Servicio	SRV-06 PagoService.listarPagos()
API / DTO	API-06 GET /api/v1/payments / DTO-06

CU-07. Consultar pagos asociados a un pedido

ID	CU-07

Nombre	Consultar pagos asociados a un pedido
Actor principal	Servicio de Pedidos o Administrador
Objetivo	Recuperar todos los intentos de pago de un pedido para reconstruir su trazabilidad.
Precondiciones	El actor está autenticado y conoce el código del pedido.
Flujo principal	1. Se envía un GET a /api/v1/payments?codigoPedido={codigoPedido}. 2. Pagos filtra los pagos por el código de pedido. 3. Pagos responde 200 OK con la lista de intentos de pago del pedido, ordenados por fecha (DTO-06). Puede haber más de uno por los reintentos (RN-05).
Flujo alternativo	*2a. Si el pedido no tiene pagos, responde 200 OK con la lista vacía.*
Resultado	Se obtiene la trazabilidad de los pagos del pedido. Se reutiliza el endpoint de listado con un filtro, para no chocar con GET /api/v1/payments/{codigoPago} de CU-04.
RN asociadas	RN-05
Entidades / relaciones	ENT-01 Pago
Secuencia	No aplica
Servicio	SRV-06 PagoService.listarPagos()
API / DTO	API-06 GET /api/v1/payments?codigoPedido= / DTO-06

CU-08. Registrar medio de pago

ID	CU-08
Nombre	Registrar medio de pago
Actor principal	Cliente, a través del Portal Web
Objetivo	Registrar un medio de pago simulado del usuario guardando solo un detalle enmascarado.
Precondiciones	El usuario está autenticado con rol CLIENTE.
Flujo principal	1. El cliente envía un POST a /api/v1/payment-methods con el tipo y el número del medio de pago (DTO-07). 2. Pagos toma el código de usuario del JWT, no del cuerpo de la solicitud. 3. Pagos enmascara el detalle sensible, guarda solo los últimos 4 dígitos y descarta el número completo (RN-08). 4. Pagos crea el MedioPago activo asociado al usuario y responde 201 Created con su código (DTO-08).
Flujo alternativo	*1a. Si el tipo de medio de pago no es soportado o los datos no son válidos, responde 400 PAYMENT_METHOD_INVALID.*
Resultado	El medio de pago queda disponible para usarse en futuros pagos.
RN asociadas	RN-08
Entidades / relaciones	ENT-03 MedioPago
Secuencia	No aplica
Servicio	SRV-07 MedioPagoService.registrarMedioPago()
API / DTO	API-07 POST /api/v1/payment-methods / DTO-07, DTO-08

CU-09. Detectar solicitudes duplicadas mediante idempotencia

ID	CU-09
Nombre	Detectar solicitudes duplicadas mediante idempotencia
Actor principal	Servicio de Pedidos
Objetivo	Evitar que una misma solicitud de pago genere más de un pago y detectar la reutilización de la clave con datos distintos.
Precondiciones	La solicitud de pago incluye la clave de control en el campo claveControl de DTO-01.
Flujo principal	1. Pedidos envía un POST a /api/v1/payments con la clave de control en el cuerpo. 2. Pagos busca si existe un pago con esa clave. 3. Si no existe, sigue el flujo de CU-01. 4. Si existe y los datos coinciden (pedido, usuario, monto y medio de pago), devuelve el pago original con 200 OK.
Flujo alternativo	*4a. Si la clave fue usada con datos distintos, responde 409 DUPLICATE_PAYMENT_REQUEST.* *4b. Si llegan dos solicitudes simultáneas con la misma clave, la restricción UNIQUE de clave_control hace fallar la segunda inserción y Pagos devuelve el pago creado por la primera.*
Resultado	Se evita la creación de pagos duplicados.
RN asociadas	RN-01, RN-05
Entidades / relaciones	ENT-01 Pago
Secuencia	SEC-01 (flujos alternativos)
Servicio	SRV-01 PagoService.iniciarPago()
API / DTO	API-01 POST /api/v1/payments / DTO-01, DTO-02

CU-10. Consultar historial de estados del pago

ID	CU-10
Nombre	Consultar historial de estados del pago
Actor principal	Administrador
Objetivo	Consultar la línea de tiempo de cambios de estado de un pago.
Precondiciones	El actor está autenticado con rol ADMINISTRADOR.
Flujo principal	1. Se envía un GET a /api/v1/payments/{codigoPago}/history. 2. Pagos recupera todos los registros de HistorialEstadoPago del pago, ordenados por fecha. 3. Pagos responde 200 OK con la línea de tiempo completa (DTO-09).
Flujo alternativo	*2a. Si el código de pago no existe, responde 404 PAYMENT_NOT_FOUND.*
Resultado	Se obtiene la trazabilidad completa del pago, útil para auditoría.
RN asociadas	RN-09
Entidades / relaciones	ENT-01 Pago, ENT-04 HistorialEstadoPago / REL-03
Secuencia	No aplica
Servicio	SRV-08 PagoService.consultarHistorial()
API / DTO	API-08 GET /api/v1/payments/{codigoPago}/history / DTO-09

Diagramas de secuencia

Presentamos los diagramas de secuencia de los cinco casos de uso principales. Todos siguen el mismo recorrido por capas: el JwtAuthenticationFilter valida el token, el controlador solo hace validación superficial (@Valid) y delega en el servicio, que aplica las reglas de negocio y lanza una PagoException con el código de error cuando algo no se cumple. El GlobalExceptionHandler (@ControllerAdvice) traduce esa excepción al error normalizado (DTO-10).

SEC-01. Iniciar pago (CU-01 y CU-09)

Incluye los tres caminos de idempotencia: clave nueva, clave repetida con los mismos datos y clave repetida con datos distintos.

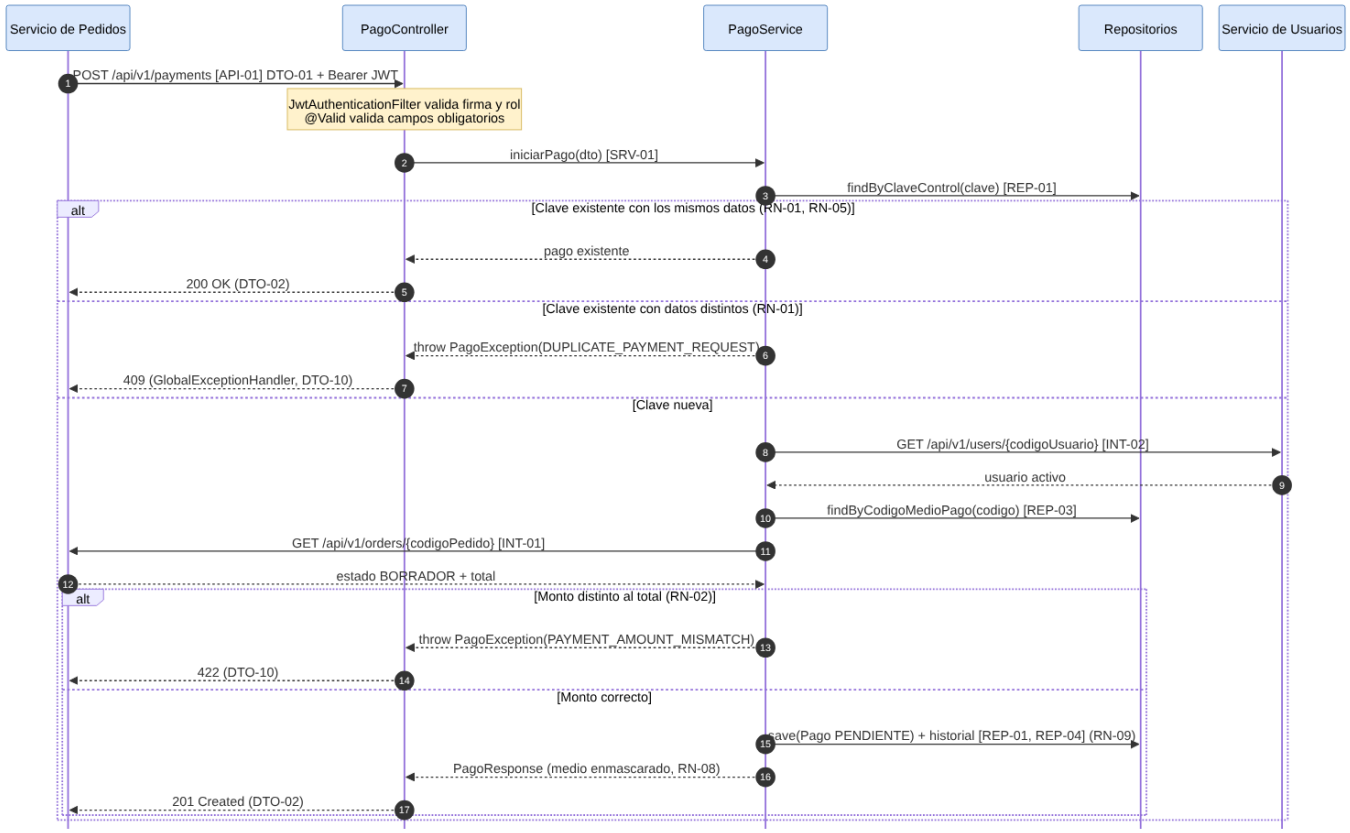


Figura 2. SEC-01. Iniciar pago (CU-01 y CU-09).

SEC-02. Autorizar pago (CU-02)

Pedidos dispara la autorización simulada y recibe el resultado en la respuesta; con un pago APROBADO confirma el pedido.

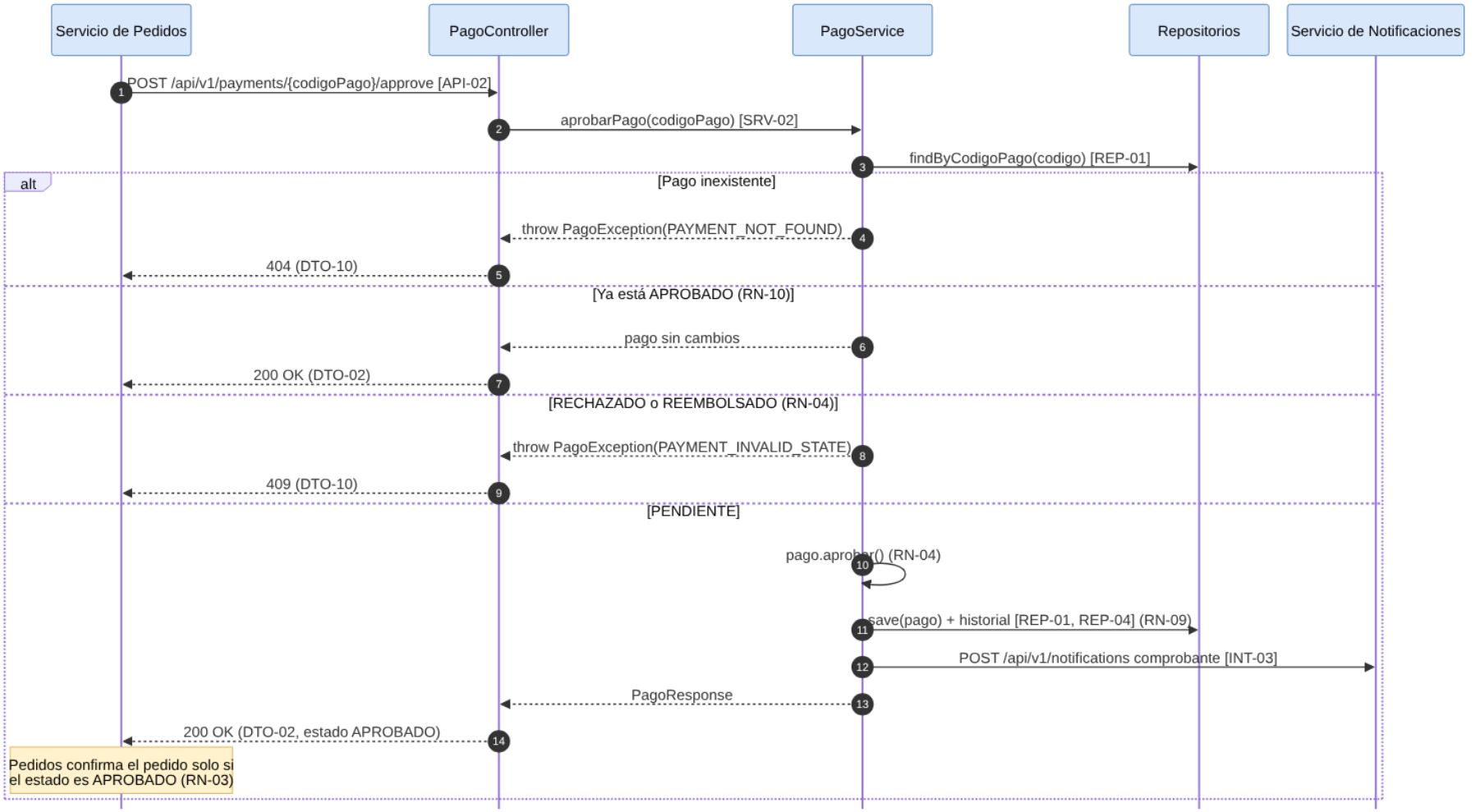


Figura 3. SEC-02. Autorizar pago (CU-02).

SEC-03. Rechazar pago (CU-03)

Repetir el rechazo sobre un pago ya RECHAZADO devuelve 200 sin cambios (RN-10).

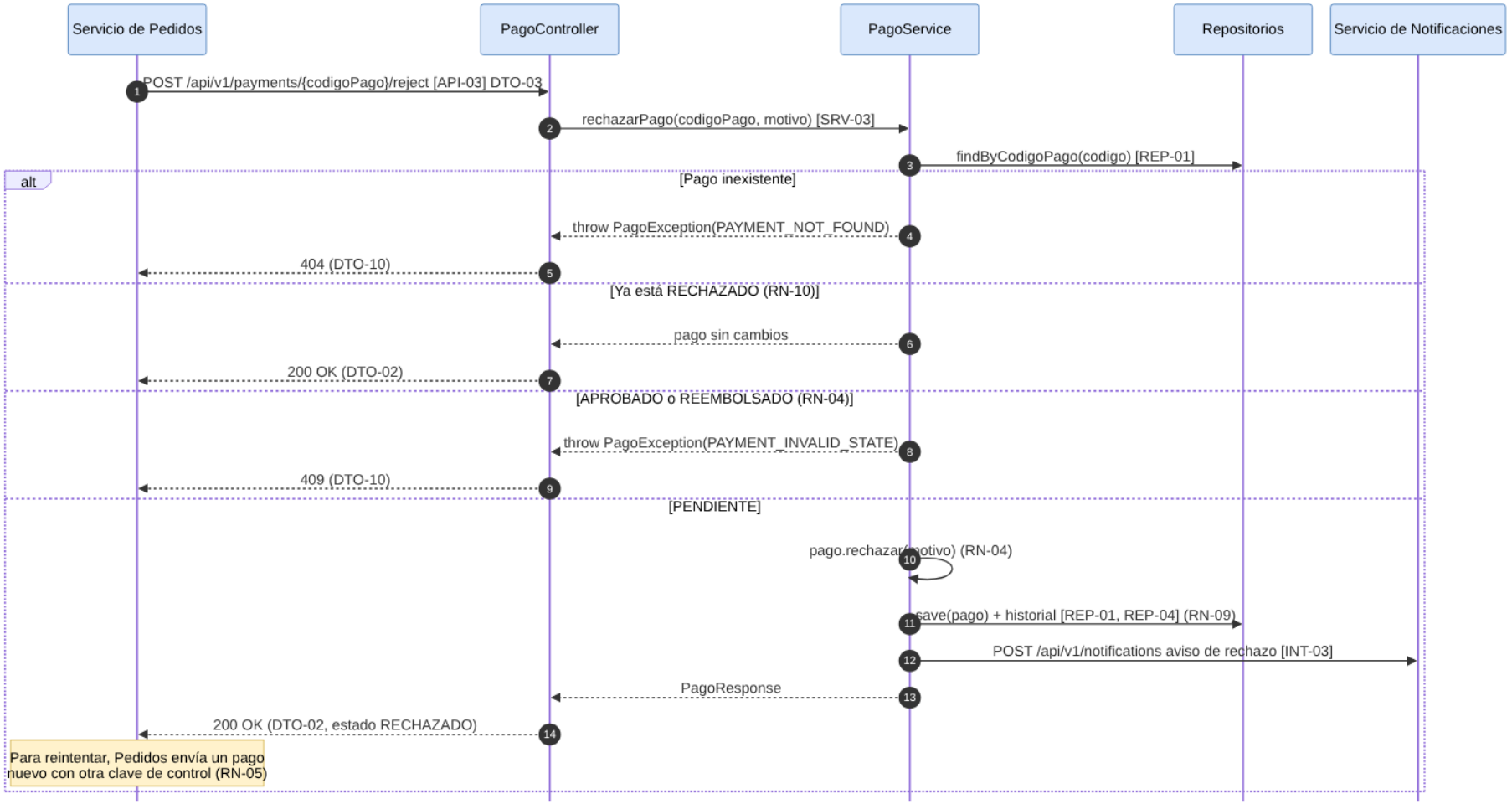


Figura 4. SEC-03. Rechazar pago (CU-03).

SEC-04. Consultar pago (CU-04)

La búsqueda es siempre por el código público; un cliente solo puede ver sus propios pagos.

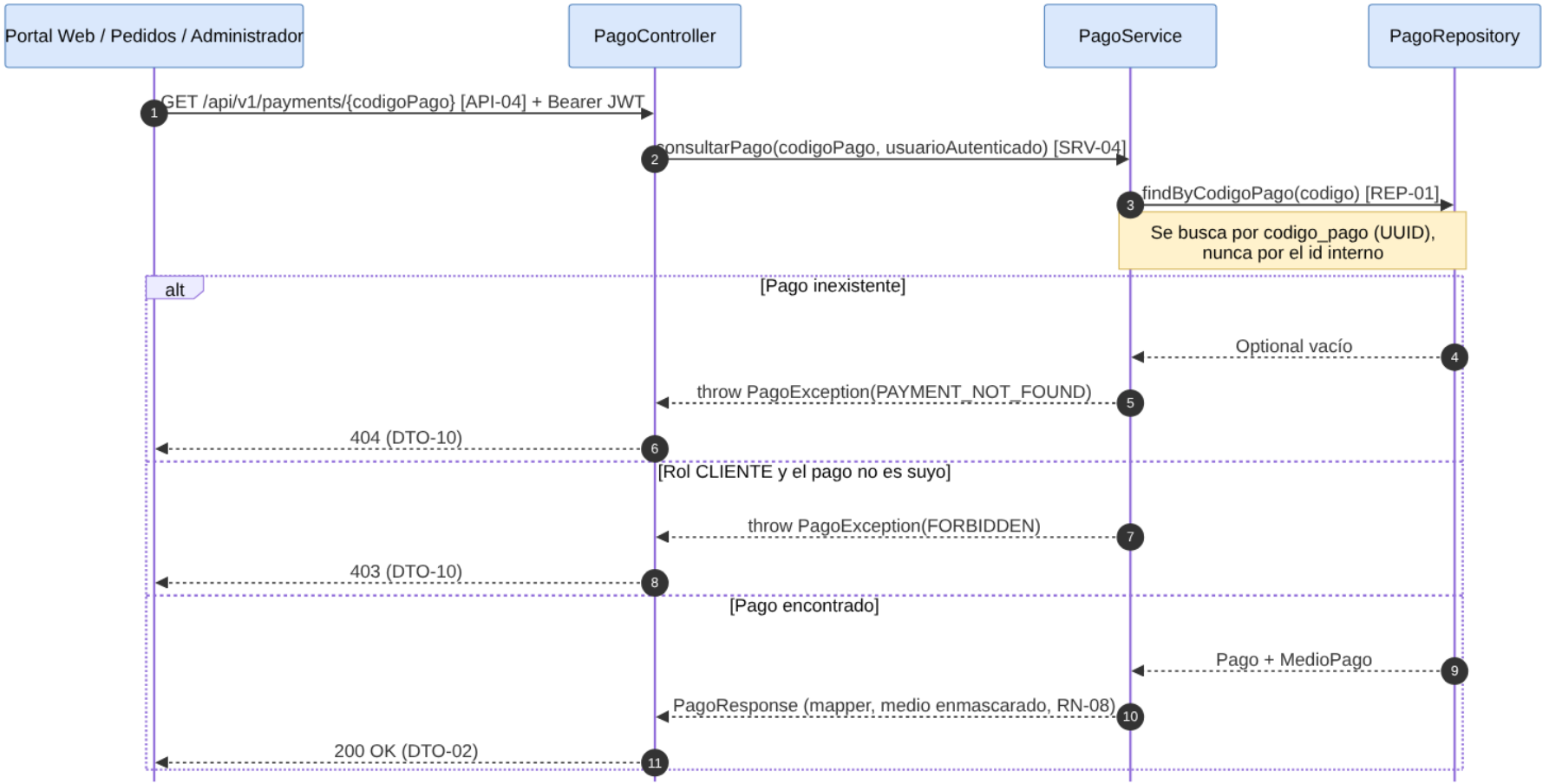


Figura 5. SEC-04. Consultar pago (CU-04).

SEC-05. Reembolsar pago (CU-05)

Un reembolso parcial deja el pago APROBADO; solo el reembolso total lo pasa a REEMBOLSADO.

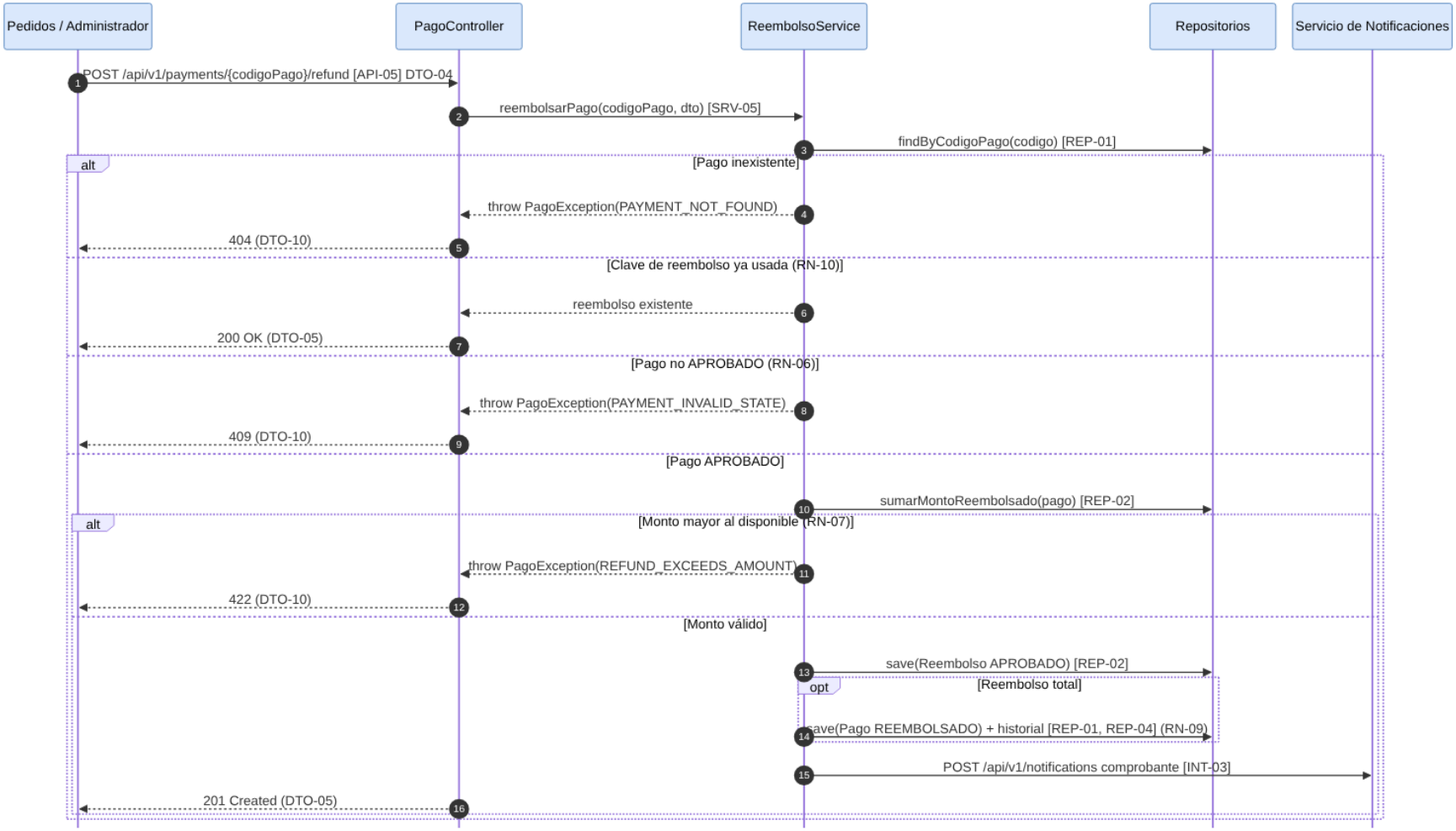


Figura 6. SEC-05. Reembolsar pago (CU-05).

Arquitectura del módulo

Documentamos la arquitectura con el modelo C4 en tres niveles: el contexto del módulo dentro de DeliveryGo, los contenedores que lo componen y los componentes internos de la API de Pagos.

Nivel 1: Contexto

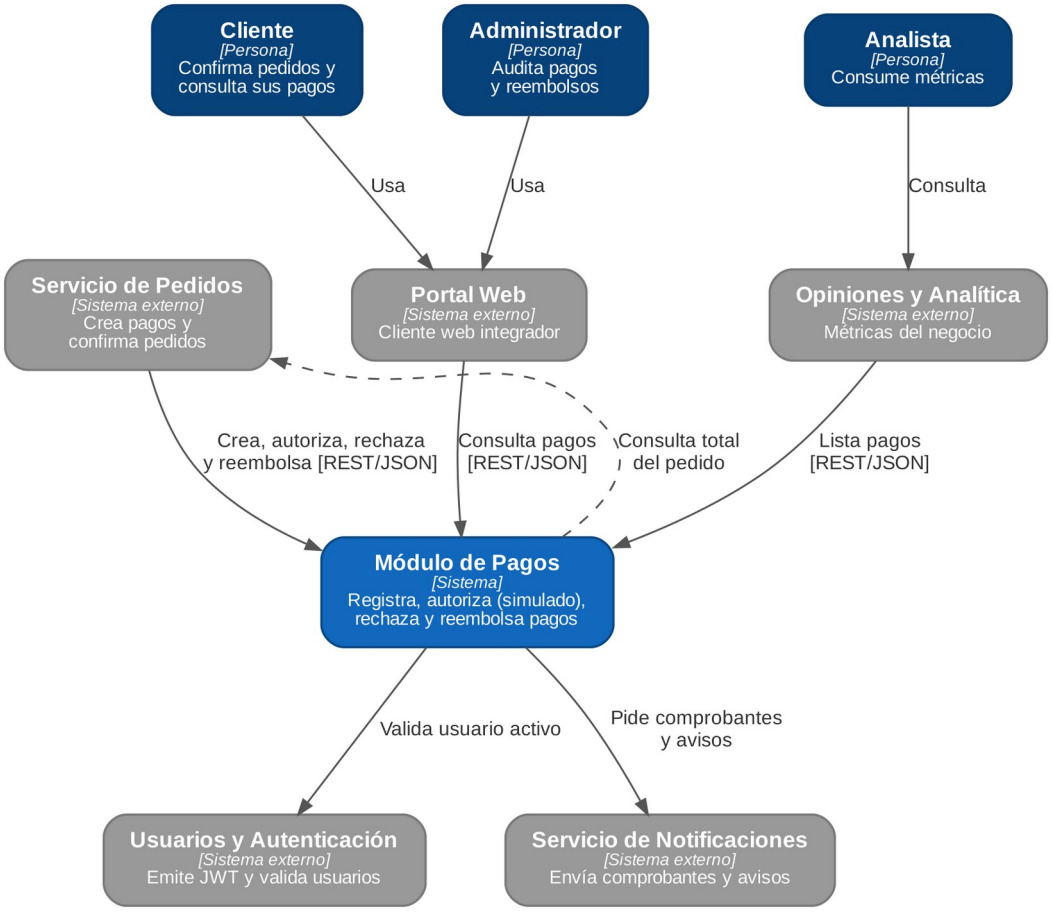


Figura 7. C4 nivel 1, contexto del Módulo de Pagos.

Nivel 2: Contenedores

El módulo tiene su propia base de datos y nunca accede a tablas de otros módulos. Toda la comunicación es REST con JSON y cada request lleva un JWT en el header Authorization: Bearer.

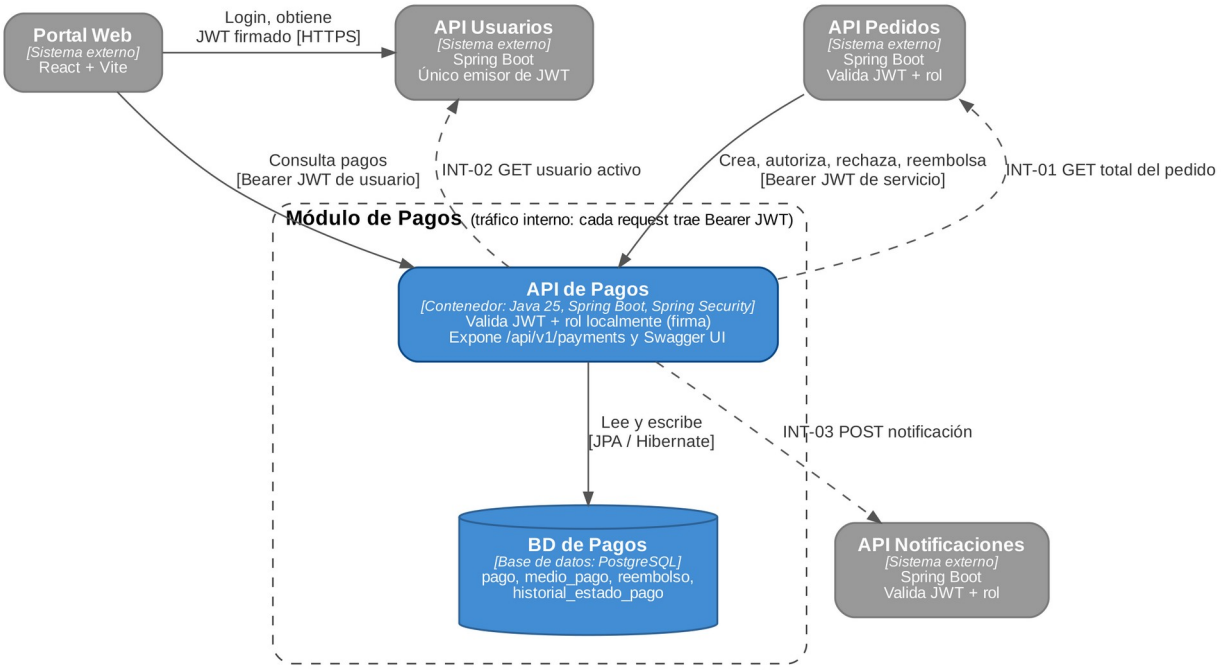


Figura 8. C4 nivel 2, contenedores del Módulo de Pagos.

Nivel 3: Componentes de la API de Pagos

La API respeta la separación controller, service y repository. Los controladores no tienen reglas de negocio, las transacciones (@Transactional) viven en la capa de servicio y los DTOs son el único formato que cruza la frontera de la API.

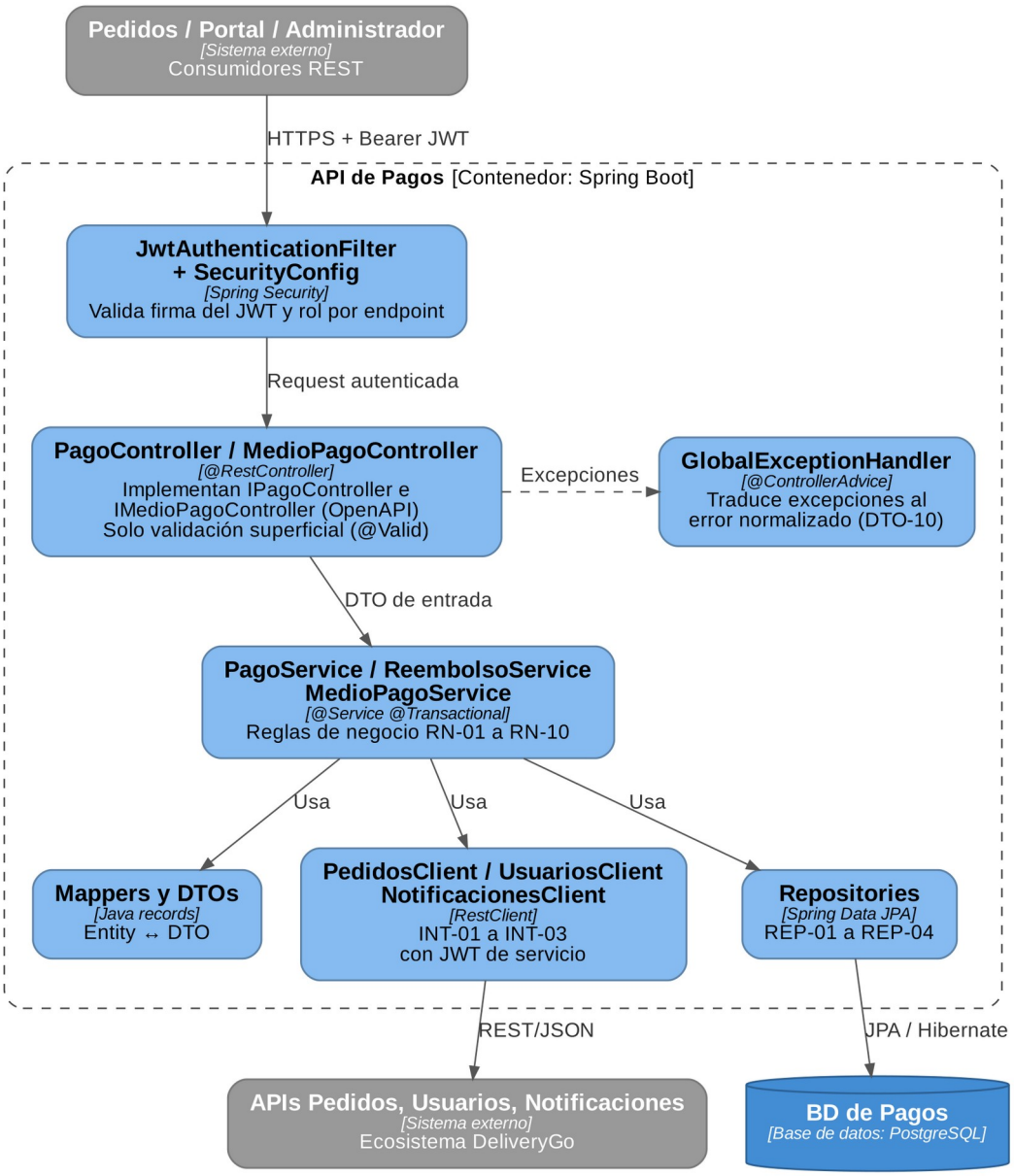


Figura 9. C4 nivel 3, componentes de la API de Pagos.

Seguridad y autorización por rol

Seguimos la arquitectura de seguridad definida para el TPO. El Portal Web se autentica contra la API de Usuarios, que es el único emisor de JWT. Desde ahí, cada API del ecosistema recibe el token en el header Authorization: Bearer y valida por su cuenta la firma y el rol, sin consultar a Usuarios en cada request. Las llamadas entre APIs (por ejemplo, Pedidos hacia Pagos) usan un JWT de servicio con permiso de lectura o de escritura que identifica a la API que llama.

Pagos solo consulta a Usuarios (INT-02) para una verificación de negocio: que el cliente que paga exista y esté activo. Esa consulta no reemplaza la validación del token. Si la solicitud no trae un JWT válido, responde 401; si el rol no tiene permiso, responde 403.

API	Endpoint	Roles autorizados
API-01	POST /api/v1/payments	Servicio de escritura (api-pedidos)
API-02	POST /api/v1/payments/{codigoPago}/approve	Servicio de escritura (api-pedidos)
API-03	POST /api/v1/payments/{codigoPago}/reject	Servicio de escritura (api-pedidos)
API-04	GET /api/v1/payments/{codigoPago}	CLIENTE (solo sus pagos), ADMINISTRADOR, servicio de lectura (api-pedidos)
API-05	POST /api/v1/payments/{codigoPago}/refund	Servicio de escritura (api-pedidos), ADMINISTRADOR
API-06	GET /api/v1/payments	ADMINISTRADOR, ANALISTA, servicio de lectura (api-pedidos, api-analitica)
API-07	POST /api/v1/payment-methods	CLIENTE
API-08	GET /api/v1/payments/{codigoPago}/history	ADMINISTRADOR

Diagrama de clases (UML)

El diagrama usa tipos de Java y referencias a objetos; los tipos de base de datos quedan en el DER. Los estados y tipos se modelan como enumeraciones. Las reglas que dependen solo del estado del pago (RN-04 y RN-06) viven en los métodos de la entidad Pago.

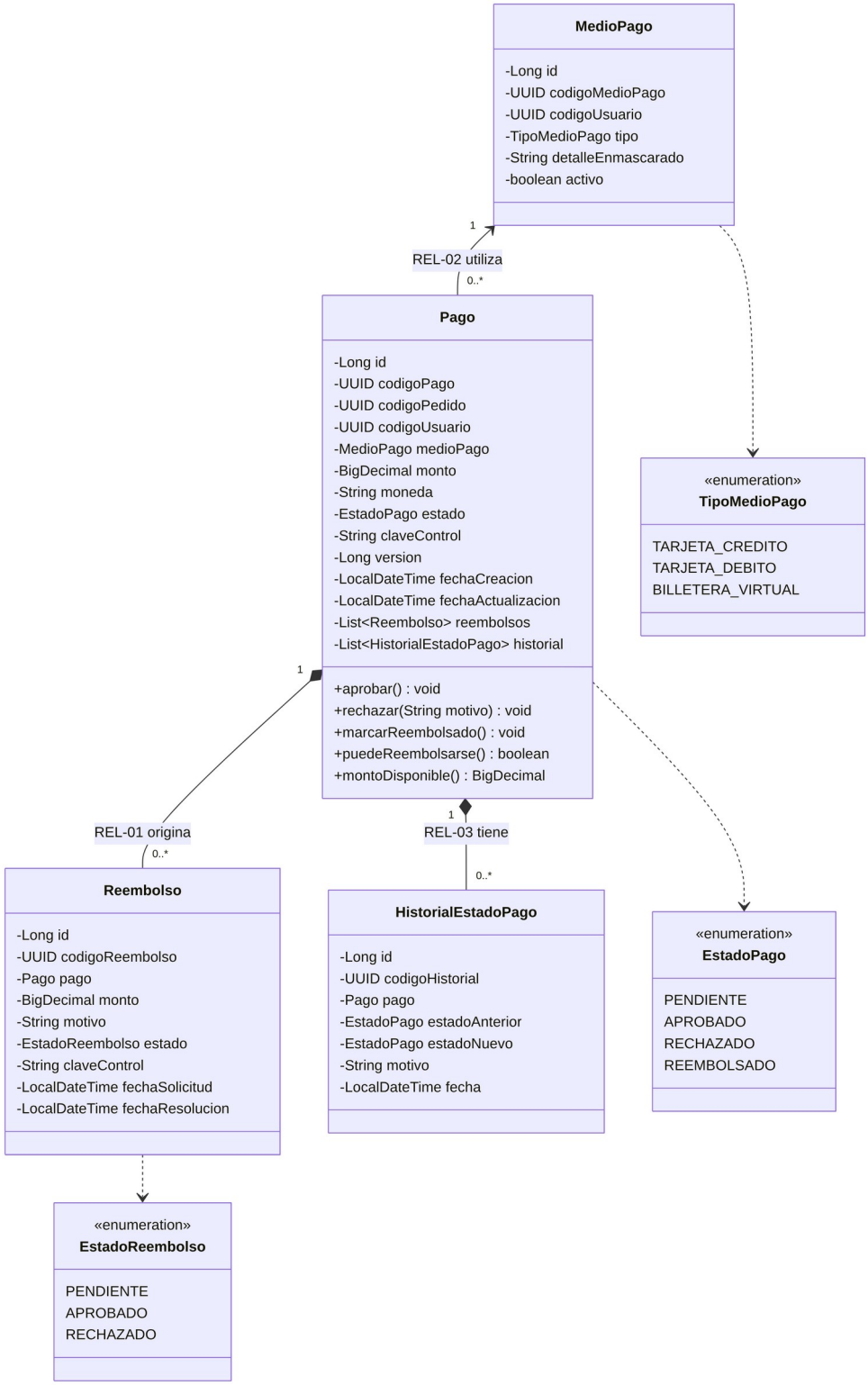


Figura 10. Diagrama de clases del Módulo de Pagos.

Reglas de negocio - Tabla

ID	Regla	CU relacionados	Dónde se implementa
RN-01	Misma clave y mismos datos devuelve el pago existente; misma clave con datos distintos, 409.	CU-01, CU-09	SRV-01 / REP-01 (UNIQUE sobre clave_control)
RN-02	El importe coincide con el total del pedido.	CU-01	SRV-01 / INT-01
RN-03	Solo un pago APROBADO hace avanzar el pedido.	CU-02, CU-03	SRV-02, SRV-03 / contrato de API-02 y API-03
RN-04	Transiciones válidas de estado.	CU-02, CU-03, CU-05	ENT-01 Pago (aprobar, rechazar, marcarReembolsado)
RN-05	Un pago rechazado se reintenta con una clave nueva.	CU-01, CU-03, CU-07, CU-09	SRV-01 / REP-01
RN-06	Solo se reembolsa un pago APROBADO.	CU-05	SRV-05 / ENT-01 (puedeReembolsarse)
RN-07	Los reembolsos no superan el monto aprobado.	CU-05	SRV-05 / REP-02
RN-08	Solo se guarda el detalle enmascarado del medio de pago.	CU-01, CU-04, CU-06, CU-08	SRV-07 / ENT-03 MedioPago / DTO-02, DTO-08
RN-09	Cada cambio de estado queda en el historial.	CU-01, CU-02, CU-03, CU-05, CU-10	SRV-01, SRV-02, SRV-03, SRV-05 / REP-04
RN-10	Repetir una operación ya aplicada no genera efectos; una incompatible responde 409.	CU-02, CU-03, CU-05	SRV-02, SRV-03, SRV-05 / REP-02

Diagrama Entidad – Relación (DER)

Cada tabla tiene un id interno autoincremental, de tipo bigint, que solo se usa para relaciones internas y búsquedas más rápidas, y una columna código, de tipo uuid con restricción UNIQUE, que es la que se expone en la API. Las referencias a otros módulos (codigo_pedido, codigo_usuario) se guardan como UUID sin clave foránea, porque cada módulo tiene su propia base de datos.

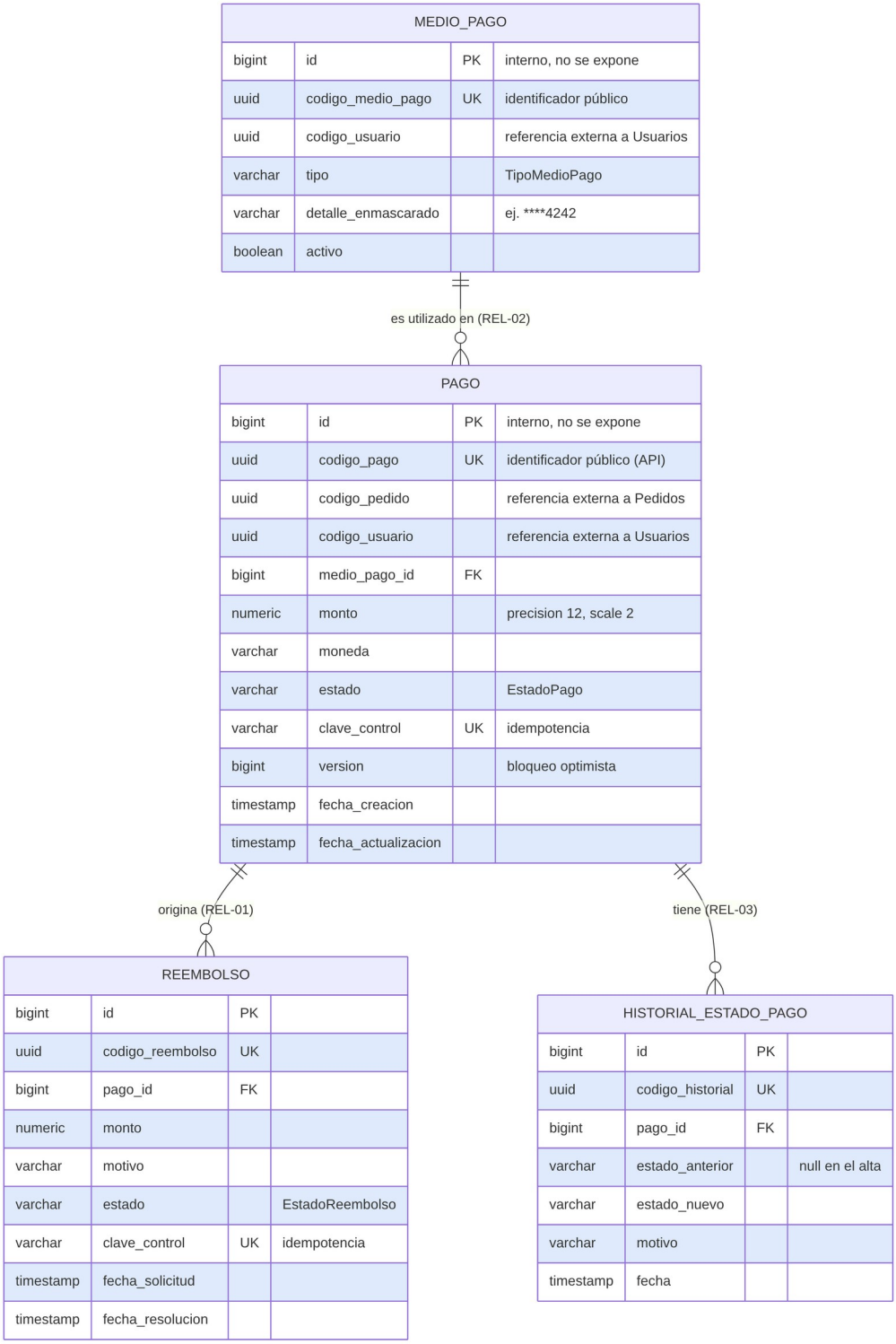


Figura 11. Diagrama Entidad – Relación del Módulo de Pagos.

Relaciones JPA

No hay relaciones @OneToOne ni @ManyToMany. El lado dueño de cada relación es la entidad que tiene la clave foránea (@JoinColumn).

ID	Relación	Multiplicidad	Mapeo JPA	Lado dueño	Fetch
REL-01	Pago – Reembolso	1 a 0..*	@OneToMany(mappedBy = "pago") en Pago / @ManyToOne en Reembolso	Reembolso (@JoinColumn pago_id)	LAZY
REL-02	MedioPago – Pago	1 a 0..*	@ManyToOne en Pago (unidireccional)	Pago (@JoinColumn medio_pago_id)	LAZY
REL-03	Pago – HistorialEstadoPago	1 a 0..*	@OneToMany(mappedBy = "pago") en Pago / @ManyToOne en HistorialEstadoPago	HistorialEstadoPago (@JoinColumn pago_id)	LAZY

Persistencia con JPA

Usamos JPA como especificación y Hibernate como implementación, sobre PostgreSQL. El código UUID de cada entidad se genera en @PrePersist y las fechas de creación y actualización se completan con @PrePersist y @PreUpdate.

Entidad	Tabla	Mapeo principal
ENT-01 Pago	pago	@Id @GeneratedValue(IDENTITY) id. codigoPago UUID único, no nulo y no actualizable. monto BigDecimal (precision 12, scale 2). estado con @Enumerated(STRING). claveControl única. @Version version. @ManyToOne(LAZY) medioPago. @OneToMany(mappedBy = "pago") reembolsos e historial.
ENT-02 Reembolso	reembolso	@Id id. codigoReembolso UUID único. @ManyToOne(LAZY) @JoinColumn("pago_id") pago. monto BigDecimal. estado con @Enumerated(STRING). claveControl única.
ENT-03 MedioPago	medio_pago	@Id id. codigoMedioPago UUID único. codigoUsuario UUID como referencia externa, sin FK. tipo con @Enumerated(STRING). detalleEnmascarado. activo.
ENT-04 HistorialEstadoPago	historial_estado_pago	@Id id. codigoHistorial UUID único. @ManyToOne(LAZY) @JoinColumn("pago_id") pago. estadoAnterior y estadoNuevo con @Enumerated(STRING). fecha.

Repositorios

Cada repositorio extiende de JpaRepository y encapsula el acceso a datos de una entidad. Las búsquedas que se exponen hacia afuera son siempre por código público; el id interno solo se usa dentro del módulo.

ID	Repositorio	Entidad	Qué resuelve	Lo usan
REP-01	PagoRepository	Pago	Busca un pago por su código público o por su clave de control (idempotencia) y lista pagos con filtros y paginación.	SRV-01 a SRV-04, SRV-06, SRV-08
REP-02	ReembolsoRepository	Reembolso	Guarda reembolsos, los busca por clave de control y suma lo ya reembolsado de un pago.	SRV-05
REP-03	MedioPagoRepository	MedioPago	Busca un medio de pago por su código público y lista los medios de un usuario.	SRV-01, SRV-07
REP-04	HistorialEstadoPagoRepository	HistorialEstadoPago	Registra los cambios de estado y devuelve la línea de tiempo de un pago ordenada por fecha.	SRV-01, SRV-02, SRV-03, SRV-05, SRV-08

Catálogos de trazabilidad

Estos catálogos definen los identificadores que usan los casos de uso y la matriz maestra. Las entidades (ENT), relaciones (REL) y repositorios (REP) están en las secciones de DER y persistencia.

Servicios

ID	Servicio y método	Responsabilidad	CU
SRV-01	PagoService.iniciarPago()	Valida idempotencia, usuario, medio de pago y monto; crea el pago PENDIENTE.	CU-01, CU-09
SRV-02	PagoService.aprobarPago()	Aplica la transición a APROBADO, registra historial y pide el comprobante.	CU-02
SRV-03	PagoService.rechazarPago()	Aplica la transición a RECHAZADO con motivo, registra historial y pide el aviso.	CU-03
SRV-04	PagoService.consultarPago()	Busca por código público y controla que un cliente solo vea sus pagos.	CU-04
SRV-05	ReembolsoService.reembolsarPago()	Valida estado, disponible e idempotencia; crea el reembolso y, si es total, marca el pago REEMBOLSADO.	CU-05
SRV-06	PagoService.listarPagos()	Lista pagos paginados con filtros (incluye el filtro por código de pedido).	CU-06, CU-07
SRV-07	MedioPagoService.registrarMedioPago()	Enmascara el detalle y registra el medio de pago del usuario autenticado.	CU-08
SRV-08	PagoService.consultarHistorial()	Devuelve la línea de tiempo de estados del pago.	CU-10

APIs

La consigna nombra el parámetro como {id}; usamos {codigoPago} porque la API expone siempre el código público y nunca el identificador interno.

ID	Método y ruta	Request	Response	Códigos HTTP
API-01	POST /api/v1/payments	DTO-01	DTO-02	201, 200, 400, 401, 403, 409, 422, 503
API-02	POST /api/v1/payments/{codigoPago}/approve	—	DTO-02	200, 401, 403, 404, 409
API-03	POST /api/v1/payments/{codigoPago}/reject	DTO-03	DTO-02	200, 400, 401, 403, 404, 409
API-04	GET /api/v1/payments/{codigoPago}	—	DTO-02	200, 401, 403, 404
API-05	POST /api/v1/payments/{codigoPago}/refund	DTO-04	DTO-05	201, 200, 400, 401, 403, 404, 409, 422
API-06	GET /api/v1/payments	Parámetros de filtro y paginación	DTO-06	200, 400, 401, 403
API-07	POST /api/v1/payment-methods	DTO-07	DTO-08	201, 400, 401, 403
API-08	GET /api/v1/payments/{codigoPago}/history	—	DTO-09	200, 401, 403, 404

DTOs

ID	Nombre	Campos
DTO-01	CrearPagoRequest	codigoPedido, codigoUsuario, codigoMedioPago, monto, moneda, claveControl

ID	Nombre	Campos
DTO-02	PagoResponse	codigoPago, codigoPedido, estado, monto, moneda, medioPago (tipo, detalleEnmascarado), fechaCreacion, fechaActualizacion
DTO-03	RechazarPagoRequest	motivo
DTO-04	ReembolsoRequest	monto, motivo, claveControl
DTO-05	ReembolsoResponse	codigoReembolso, codigoPago, monto, estado, montoDisponible, fechaSolicitud, fechaResolucion
DTO-06	PaginaPagosResponse	content (lista de DTO-02), page, size, totalElements, totalPages
DTO-07	MedioPagoRequest	tipo, numero (solo en tránsito, nunca se guarda completo)
DTO-08	MedioPagoResponse	codigoMedioPago, tipo, detalleEnmascarado, activo
DTO-09	HistorialEstadoResponse	estadoAnterior, estadoNuevo, motivo, fecha
DTO-10	ErrorResponse	timestamp, status, code, message, path, correlationId

Integraciones

Todas las integraciones son sincrónicas por REST, usan un JWT de servicio, propagan el header X-Correlation-Id y tienen timeout. El resultado de la autorización le llega a Pedidos en la respuesta de API-02 y API-03, por lo que no hace falta una llamada de Pagos hacia Pedidos para avisarle.

ID	Módulo	Endpoint	Para qué	Si falla
INT-01	Pedidos	GET /api/v1/orders/{codigoPedido}	Obtener el estado y el total del pedido (RN-02).	503; no se crea el pago.
INT-02	Usuarios	GET /api/v1/users/{codigoUsuario}	Verificar que el usuario exista y esté activo.	503; no se crea el pago.
INT-03	Notificaciones	POST /api/v1/notifications	Pedir comprobantes y avisos de aprobación, rechazo y reembolso.	Se registra en el log; la operación de pago no se revierte.

Matriz maestra de trazabilidad

Cada fila permite seguir un caso de uso desde sus reglas de negocio hasta la persistencia y las integraciones. Los CU-06 a CU-10 no tienen diagrama de secuencia propio porque no son parte de los cinco casos principales; CU-09 se ve en los flujos alternativos de SEC-01.

CU	RN	SEC	SRV	API	DTO	ENT / REL	REP	INT
CU-01	RN-01, 02, 05, 08, 09	SEC-01	SRV-01	API-01	DTO-01, 02	ENT-01, 03, 04 / REL-02, 03	REP-01, 03, 04	INT-01, 02
CU-02	RN-03, 04, 09, 10	SEC-02	SRV-02	API-02	DTO-02	ENT-01, 04 / REL-03	REP-01, 04	INT-03
CU-03	RN-03, 04, 05, 09, 10	SEC-03	SRV-03	API-03	DTO-03, 02	ENT-01, 04 / REL-03	REP-01, 04	INT-03
CU-04	RN-08	SEC-04	SRV-04	API-04	DTO-02	ENT-01, 03 / REL-02	REP-01	—
CU-05	RN-04, 06, 07, 09, 10	SEC-05	SRV-05	API-05	DTO-04, 05	ENT-01, 02, 04 / REL-01, 03	REP-01, 02, 04	INT-03
CU-06	RN-08	—	SRV-06	API-06	DTO-06	ENT-01, 03 / REL-02	REP-01	—
CU-07	RN-05	—	SRV-06	API-06	DTO-06	ENT-01	REP-01	—
CU-08	RN-08	—	SRV-07	API-07	DTO-07, 08	ENT-03	REP-03	—
CU-09	RN-01, 05	SEC-01	SRV-01	API-01	DTO-01, 02	ENT-01	REP-01	—
CU-10	RN-09	—	SRV-08	API-08	DTO-09	ENT-01, 04 / REL-03	REP-04	—

Manejo de errores

Todos los errores se devuelven con el formato normalizado del ecosistema (DTO-10), armado por el GlobalExceptionHandler.

Campo	Descripción
timestamp	Fecha y hora del error, en formato ISO-8601.
status	Código HTTP asociado al error.
code	Código de error propio del módulo, según el listado de códigos de error del módulo.
message	Mensaje descriptivo pensado para quien consume la API.
path	Endpoint donde ocurrió el error.
correlationId	Identificador de correlación, tomado del header X-Correlation-Id, que sirve para seguir el rastro entre módulos.

Códigos de error del módulo

Código	HTTP	Situación
PAYMENT_NOT_FOUND	404	No existe un pago con el código informado.
PAYMENT_AMOUNT_MISMATCH	422	El monto del pago no coincide con el total del pedido.
PAYMENT_INVALID_STATE	409	Se intenta una operación que no está permitida para el estado actual del pago.
DUPLICATE_PAYMENT_REQUEST	409	La clave de control ya se usó antes con datos distintos.
REFUND_EXCEEDS_AMOUNT	422	El monto de reembolso solicitado supera lo disponible.
PAYMENT_METHOD_INVALID	400	El tipo o los datos del medio de pago no son válidos, o el medio no pertenece al usuario.
PAYMENT_USER_INVALID	422	El usuario no existe o está inactivo.
ORDER_NOT_PAYABLE	422	El pedido no existe o no está pendiente de pago.
VALIDATION_ERROR	400	Faltan campos obligatorios o tienen un formato inválido.
UNAUTHORIZED	401	La solicitud no trae un JWT válido.
FORBIDDEN	403	El JWT es válido pero el rol no tiene permiso para la operación o el recurso.
EXTERNAL_SERVICE_UNAVAILABLE	503	Pedidos o Usuarios no respondieron a tiempo.
INTERNAL_ERROR	500	Error no controlado. Nunca se exponen detalles internos ni stack traces.

Decisiones técnicas

ID	Decisión
DEC-01	Cada tabla tiene un id interno bigint, que no se expone, y un código UUID único que es el que se usa en la API y en las referencias a otros módulos.
DEC-02	La clave de control viaja en el cuerpo de la solicitud (claveControl) y tiene restricción UNIQUE, que también cubre solicitudes simultáneas.
DEC-03	La autorización es simulada: Pedidos llama a approve o reject y recibe el resultado en la respuesta, sin callback de Pagos hacia Pedidos.
DEC-04	Usuarios es el único emisor de JWT. Pagos valida localmente la firma y el rol en cada request; entre APIs se usa un JWT de servicio.
DEC-05	Pago usa bloqueo optimista (@Version) para que dos operaciones simultáneas sobre el mismo pago no generen dos transiciones.
DEC-06	Los estados y tipos se modelan como enums y se persisten con @Enumerated(STRING). Los montos usan BigDecimal y numeric(12,2).
DEC-07	Todas las relaciones son LAZY. Cuando hace falta el medio de pago se usa una consulta con fetch join, para evitar el problema N+1.
DEC-08	Si Notificaciones falla, la operación de pago no se revierte; el error queda registrado en el log.
DEC-09	Los controladores implementan interfaces (IPagoController, IMedioPagoController) con las anotaciones de OpenAPI y no tienen reglas de negocio.

Backlog inicial

ID	Tarea	Prioridad	Estado	Entrega
BL-01	Análisis y diseño del módulo (este documento)	Alta	Hecho	1
BL-02	Repositorio Git con ramas main y develop, README y estructura de carpetas	Alta	Pendiente	1
BL-03	Acordar contratos con Pedidos, Usuarios y Notificaciones (INT-01 a INT-03)	Alta	Pendiente	1 y 2
BL-04	Proyecto Spring Boot con Java 25, PostgreSQL y docker-compose	Alta	Pendiente	2
BL-05	Entidades, enums y relaciones JPA (ENT-01 a ENT-04)	Alta	Pendiente	2
BL-06	Repositorios REP-01 a REP-04	Alta	Pendiente	2
BL-07	SRV-01: iniciar pago con idempotencia	Alta	Pendiente	2
BL-08	SRV-02 y SRV-03: aprobar y rechazar con historial	Alta	Pendiente	2
BL-09	SRV-04: consultar pago	Alta	Pendiente	2
BL-10	SRV-05: reembolsos totales y parciales	Alta	Pendiente	2
BL-11	Seguridad: JwtAuthenticationFilter y reglas por rol	Alta	Pendiente	2
BL-12	GlobalExceptionHandler y errores normalizados	Alta	Pendiente	2
BL-13	OpenAPI y Swagger con interfaces I*Controller	Alta	Pendiente	2
BL-14	Clientes HTTP de INT-01 a INT-03, con mocks mientras no estén los otros módulos	Alta	Pendiente	2
BL-15	SRV-06: listado paginado con filtros	Media	Pendiente	2
BL-16	SRV-07: registro de medios de pago	Media	Pendiente	2

ID	Tarea	Prioridad	Estado	Entrega
BL-17	SRV-08: historial de estados	Media	Pendiente	2
BL-18	Datos de prueba y colección de Postman en el repositorio	Media	Pendiente	2
BL-19	Tests de las reglas de negocio RN-01 a RN-10	Media	Pendiente	2